UNIVERSITAS
DIAN NUSANTARA

Struktur Data dan Algoritma

Pengantar Struktur Data dan Algoritma

Fakultas : FTI

Program Studi : TEKNIK INFORMATIKA

Tatap Muka

01

Kode Matakuliah : 412523301

Disusun oleh : Riad Sahara, S.SI., M.T.

DESKRIPSI MATERI PERTEMUAN 1

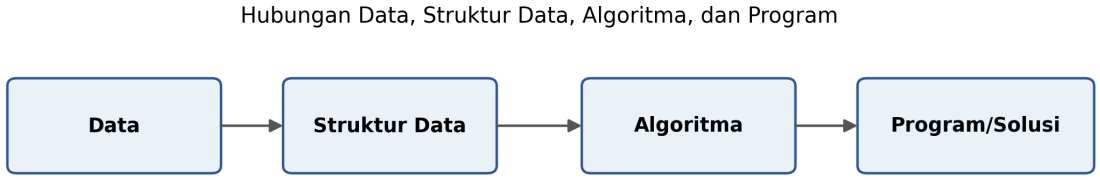
Materi Pertemuan 1	**Pengantar Struktur Data dan Algoritma**
Deskripsi Materi	Materi membahas konsep dasar struktur data dan algoritma, Abstract Data Type (ADT), klasifikasi struktur data, operasi dasar, pengantar efisiensi algoritma, serta persiapan lingkungan dan penggunaan Python sebagai alat implementasi.
Sub Capaian Pembelajaran Mata Kuliah (Sub CPMK)	Luaran yang diharapkan setelah mengikuti materi pada pertemuan ini: • Mahasiswa mampu menjelaskan hubungan data, struktur data, algoritma, dan program. • Mahasiswa mampu membedakan struktur data linear dan non-linear secara konseptual. • Mahasiswa mampu menjelaskan konsep Abstract Data Type (ADT) dan operasi dasar pada struktur data. • Mahasiswa memahami makna awal efisiensi waktu dan ruang serta notasi Big-O secara intuitif. • Mahasiswa mampu menyiapkan lingkungan praktik Python menggunakan Google Colab dan menjalankan kode sederhana. • Mahasiswa mampu membaca dan menjelaskan sintaks Python dasar yang digunakan dalam implementasi struktur data.
Deskripsi Tugas	Latihan formatif berupa kuis konsep dan praktik Python pada Google Colab. Mahasiswa membuat notebook, menjalankan contoh kode, memodifikasi data, serta mencoba fungsi pencarian linear sederhana.
Kontrak Perkuliahan	**Ketentuan perkuliahan:** 1. Mahasiswa membaca atau mengunduh modul sebelum/sesudah perkuliahan. 2. Mahasiswa mengikuti diskusi, latihan, dan aktivitas praktik yang diberikan. 3. Python digunakan sebagai alat implementasi; fokus utama tetap pada konsep struktur data dan algoritma. 4. Komposisi penilaian mengikuti RPS resmi dan ketentuan kelas yang berlaku.

Pertemuan 1
Pengantar Struktur Data dan Algoritma

1.1. Mengapa Struktur Data dan Algoritma Penting?

Program komputer bekerja dengan data. Data tersebut dapat berupa angka, teks, daftar mahasiswa, transaksi, lokasi pada peta, hubungan pertemanan, maupun informasi lain yang harus disimpan dan diproses. Ketika ukuran data kecil, hampir semua cara penyimpanan terlihat cukup. Namun, ketika jumlah data bertambah atau operasi harus dilakukan berulang kali, cara mengorganisasi data menjadi sangat menentukan. Di sinilah struktur data dan algoritma memiliki peran utama.

Struktur data adalah cara mengorganisasi dan menyimpan data agar operasi tertentu dapat dilakukan secara efektif. Algoritma adalah urutan langkah yang terdefinisi untuk menyelesaikan suatu masalah. Keduanya tidak berdiri sendiri: struktur data menyediakan bentuk penyimpanan, sedangkan algoritma menentukan cara data tersebut diproses.



Gambar 1.1 Hubungan data, struktur data, algoritma, dan program

Sumber: Diolah oleh penulis.

Sebagai contoh, sebuah aplikasi kampus harus mencari data mahasiswa berdasarkan NIM. Jika data disimpan sebagai daftar biasa, pencarian dapat dilakukan dengan memeriksa elemen satu per satu. Cara ini sederhana, tetapi semakin lambat ketika data bertambah. Dengan struktur data dan algoritma yang tepat, pencarian dapat dibuat jauh lebih efisien. Karena itu, tujuan mata kuliah ini bukan sekadar membuat program yang “berjalan”, tetapi memahami mengapa suatu solusi bekerja dan bagaimana memilih solusi yang sesuai.

Tujuan Pembelajaran pada Pertemuan Ini

- Memahami hubungan antara data, struktur data, algoritma, dan program.
- Mengenali klasifikasi dasar struktur data yang akan dipelajari selama satu semester.
- Memahami konsep ADT, operasi dasar, serta pengantar efisiensi algoritma.

• Menempatkan Python sebagai alat implementasi, bukan sebagai tujuan utama pembelajaran.

1.2. Konsep Data, Struktur Data, dan Algoritma

1.2.1 Data

Data adalah representasi fakta, nilai, simbol, atau objek yang dapat disimpan dan diproses oleh komputer. Dalam program sederhana, satu nilai dapat disimpan pada sebuah variabel. Dalam aplikasi nyata, data biasanya berjumlah banyak dan saling berhubungan sehingga memerlukan organisasi yang lebih terstruktur.

Contoh data pada sistem akademik meliputi NIM, nama mahasiswa, nilai mata kuliah, daftar mata kuliah yang diambil, serta hubungan prasyarat antar-mata kuliah. Data yang sama dapat membutuhkan cara penyimpanan berbeda bergantung pada operasi yang paling sering dilakukan.

1.2.2 Struktur Data

Struktur data adalah metode untuk mengorganisasi sekumpulan data beserta hubungan antarelemen agar proses seperti penyimpanan, pencarian, penambahan, penghapusan, dan pengurutan dapat dilakukan dengan cara yang terkontrol. Struktur data yang berbeda memiliki karakteristik dan biaya operasi yang berbeda.

1.2.3 Algoritma

Algoritma merupakan rangkaian langkah logis, terurut, dan berhingga untuk mengubah masukan menjadi keluaran yang diharapkan. Suatu algoritma yang baik tidak hanya menghasilkan jawaban yang benar, tetapi juga cukup efisien dan mudah dipahami. Dalam mata kuliah ini, algoritma akan selalu dibahas bersama struktur data yang digunakan.

Konsep	Pertanyaan Utama	Contoh
Data	Apa yang disimpan?	NIM, nilai, nama, lokasi
Struktur Data	Bagaimana data diorganisasi?	Array, linked list, stack, queue, tree, graph
Algoritma	Bagaimana data diproses?	Searching, sorting, traversal
Program	Bagaimana solusi dijalankan komputer?	Implementasi Python

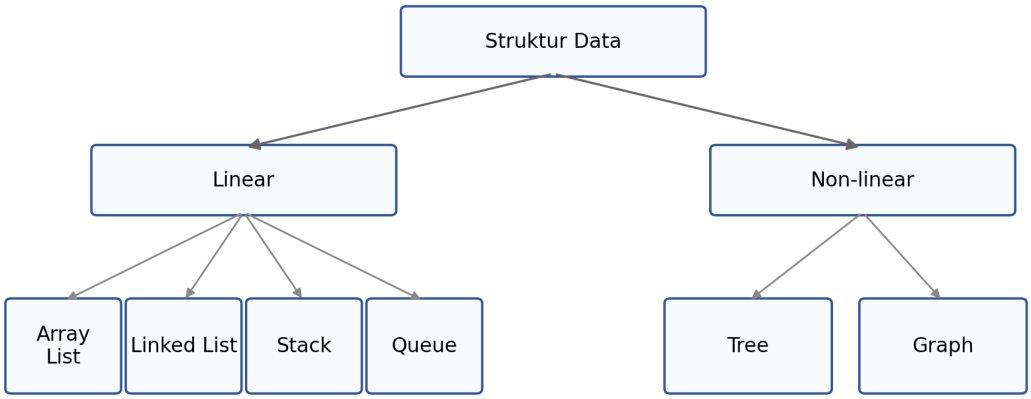
1.2.4 Program Benar Belum Tentu Efisien

Dua program dapat menghasilkan keluaran yang sama tetapi membutuhkan waktu dan memori yang berbeda. Misalnya, dua algoritma pencarian sama-sama berhasil menemukan NIM mahasiswa. Algoritma pertama memeriksa seluruh data, sedangkan algoritma kedua memanfaatkan data yang sudah terurut. Perbedaan cara tersebut akan terlihat nyata ketika

ukuran data membesar. Inilah alasan analisis algoritma menjadi bagian penting dari mata kuliah Struktur Data dan Algoritma.

1.3. Klasifikasi Dasar Struktur Data

Struktur data dapat dikelompokkan dengan berbagai cara. Untuk pembelajaran dasar, klasifikasi yang paling mudah digunakan adalah struktur data linear dan non-linear. Klasifikasi ini membantu mahasiswa memahami bentuk hubungan antarelemen sebelum mempelajari implementasinya.



Klasifikasi sederhana yang akan digunakan sepanjang mata kuliah

Gambar 1.2 Klasifikasi sederhana struktur data

Sumber: Diolah oleh penulis.

1.3.1 Struktur Data Linear

Pada struktur data linear, elemen dipandang tersusun dalam urutan tertentu. Setiap elemen umumnya memiliki posisi sebelum atau sesudah elemen lain. Contohnya adalah array, linked list, stack, dan queue. Walaupun keempatnya sama-sama linear, aturan akses dan operasi yang tersedia dapat berbeda.

- Array atau list: elemen dapat diakses menggunakan indeks.
- Linked list: elemen tersusun sebagai node yang saling terhubung.
- Stack: akses mengikuti prinsip LIFO, yaitu elemen terakhir masuk menjadi elemen pertama keluar.
- Queue: akses mengikuti prinsip FIFO, yaitu elemen pertama masuk menjadi elemen pertama keluar.

1.3.2 Struktur Data Non-linear

Pada struktur data non-linear, hubungan antar-elemen tidak harus membentuk satu urutan lurus. Tree digunakan untuk merepresentasikan hubungan hierarkis, sedangkan graph dapat merepresentasikan hubungan yang lebih umum seperti jaringan jalan, hubungan pengguna, atau koneksi antarkomputer.

1.3.3 Struktur Data Statis dan Dinamis

Klasifikasi lain melihat kemampuan ukuran struktur data untuk berubah. Struktur statis memiliki ukuran yang ditentukan dan relatif tetap, sedangkan struktur dinamis dapat bertambah atau berkurang selama program berjalan. Dalam Python, list bersifat dinamis sehingga ukurannya dapat berubah. Namun, secara konseptual mahasiswa tetap perlu memahami array statis karena banyak pembahasan kompleksitas dan implementasi struktur data berangkat dari konsep tersebut.

1.4. Abstract Data Type (ADT)

Abstract Data Type (ADT) adalah model abstrak yang mendefinisikan jenis data beserta operasi yang dapat dilakukan terhadap data tersebut tanpa mengharuskan pengguna mengetahui detail implementasi internalnya. Dengan kata lain, ADT berfokus pada apa yang dapat dilakukan, sedangkan implementasi berfokus pada bagaimana operasi tersebut diwujudkan.



Gambar 1.3 Konsep Abstract Data Type (ADT)

Sumber: Diolah oleh penulis.

1.4.1 Contoh ADT Stack

Sebuah stack dapat didefinisikan melalui sejumlah operasi dasar, misalnya push untuk menambahkan elemen, pop untuk mengambil elemen teratas, peek untuk melihat elemen

teratas tanpa menghapusnya, dan is_empty untuk memeriksa apakah stack kosong. Definisi ini belum menentukan apakah stack disimpan menggunakan array, linked list, atau fasilitas bawaan bahasa pemrograman.

Operasi	Makna
push(x)	Menambahkan elemen x ke bagian teratas stack.
pop()	Menghapus dan mengembalikan elemen teratas.
peek()	Melihat elemen teratas tanpa menghapusnya.
is_empty()	Memeriksa apakah stack tidak memiliki elemen.

1.4.2 Mengapa Konsep ADT Penting?

ADT memisahkan cara menggunakan suatu struktur data dari cara struktur tersebut dibuat. Pemisahan ini membantu perancangan program yang modular. Ketika implementasi internal diubah, bagian program yang menggunakan ADT tidak harus ikut berubah selama kontrak operasinya tetap sama. Konsep ini akan terus muncul pada pembahasan stack, queue, tree, graph, dan struktur data lainnya.

1.5. Operasi Dasar pada Struktur Data

Walaupun setiap struktur data mempunyai aturan tersendiri, terdapat beberapa operasi yang sering muncul. Memahami operasi ini penting karena perbandingan struktur data biasanya dilakukan berdasarkan kebutuhan operasi yang dominan pada suatu masalah.

Operasi	Deskripsi	Contoh Pertanyaan
Traversal	Mengunjungi elemen secara sistematis.	Bagaimana membaca semua data?
Access	Mengambil elemen tertentu.	Bagaimana mengambil elemen indeks ke-5?
Search	Mencari elemen berdasarkan nilai/kunci.	Apakah NIM tertentu tersedia?
Insert	Menambahkan elemen baru.	Bagaimana menambah data di awal?
Delete	Menghapus elemen.	Bagaimana menghapus data tertentu?
Update	Mengubah nilai elemen.	Bagaimana memperbarui nilai mahasiswa?
Sort	Mengurutkan elemen.	Bagaimana mengurutkan nilai dari kecil ke besar?

1.5.1 Operasi yang Sama, Biaya yang Berbeda

Satu operasi dapat memiliki biaya yang berbeda pada struktur data yang berbeda. Mengambil elemen berdasarkan indeks pada array umumnya sangat cepat karena posisi elemen dapat dihitung langsung. Sebaliknya, pada linked list, program biasanya harus menelusuri node dari awal sampai posisi yang diinginkan. Karena itu, pemilihan struktur data tidak dapat dipisahkan dari jenis operasi yang paling sering dilakukan.

1.5.2 Contoh Sederhana

Bayangkan aplikasi antrean layanan akademik. Jika kebutuhan utamanya adalah melayani mahasiswa sesuai urutan kedatangan, struktur queue lebih sesuai dibanding stack. Jika kebutuhan berubah menjadi fitur “undo” pada editor, stack lebih natural karena aksi terakhir harus dibatalkan lebih dahulu. Struktur data dipilih karena karakteristik operasinya cocok dengan pola masalah.

Prinsip yang digunakan sepanjang mata kuliah ini adalah: pahami masalah terlebih dahulu, identifikasi operasi utama, lalu pilih struktur data dan algoritma yang mendukung operasi tersebut dengan baik.

1.6. Pengantar Efisiensi Algoritma

Kebenaran merupakan syarat pertama sebuah algoritma, tetapi bukan satu-satunya pertimbangan. Ketika beberapa algoritma sama-sama menghasilkan jawaban yang benar, kita perlu menilai efisiensinya. Dua sumber daya yang paling umum dianalisis adalah waktu eksekusi dan penggunaan memori.

1.6.1 Kompleksitas Waktu

Kompleksitas waktu menggambarkan bagaimana jumlah operasi algoritma bertambah ketika ukuran masukan meningkat. Analisis ini tidak sama dengan mengukur detik secara langsung karena waktu nyata dipengaruhi perangkat keras, sistem operasi, compiler/interpreter, serta kondisi saat program dijalankan. Kompleksitas memberi gambaran pertumbuhan yang lebih umum.

1.6.2 Kompleksitas Ruang

Kompleksitas ruang menggambarkan kebutuhan memori tambahan selama algoritma berjalan. Sebuah solusi dapat sangat cepat tetapi membutuhkan memori lebih besar, atau sebaliknya. Pada praktiknya, pemilihan solusi sering melibatkan trade-off antara waktu dan ruang.

1.6.3 Ukuran Input

Simbol n biasanya digunakan untuk menyatakan ukuran input. Pada daftar mahasiswa, n dapat berarti jumlah mahasiswa. Pada matriks, ukuran input dapat melibatkan jumlah baris dan kolom. Sebelum menganalisis algoritma, kita harus menentukan terlebih dahulu apa yang dimaksud dengan ukuran input.

Situasi	Ukuran Input yang Mungkin Digunakan

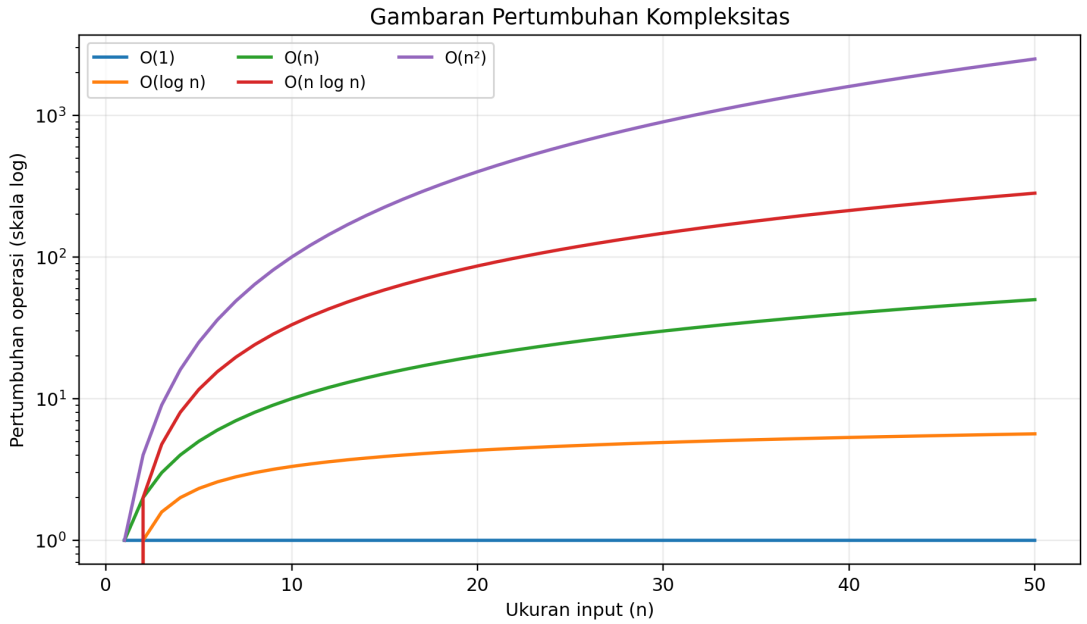
Mencari NIM dalam daftar mahasiswa	n = jumlah mahasiswa
Mengurutkan daftar nilai	n = jumlah nilai
Memproses matriks	n dan m = jumlah baris dan kolom
Menelusuri graph	V = jumlah vertex, E = jumlah edge

1.6.4 Analisis Teoretis dan Eksperimen

Pada mata kuliah ini, mahasiswa akan menggunakan dua pendekatan. Pertama, analisis teoretis menggunakan notasi kompleksitas. Kedua, eksperimen menggunakan Python untuk melihat kecenderungan waktu eksekusi pada ukuran data yang berbeda. Eksperimen membantu membangun intuisi, sedangkan analisis teoretis memberikan penjelasan yang lebih umum.

1.7. Mengenal Notasi Big-O secara Intuitif

Big-O digunakan untuk menyatakan batas atas pertumbuhan kebutuhan komputasi ketika ukuran input membesar. Pada tahap awal, mahasiswa tidak perlu memperlakukan Big-O sebagai rumus yang harus dihafal, melainkan sebagai bahasa untuk membandingkan pola pertumbuhan algoritma.



Gambar 1.4 Gambaran pertumbuhan beberapa kelas kompleksitas

Sumber: Diolah oleh penulis.

Kompleksitas	Intuisi	Contoh Awal
O(1)	Jumlah operasi relatif tetap.	Mengakses elemen array berdasarkan indeks.
O(log n)	Masalah berkurang secara signifikan setiap langkah.	Binary search pada data terurut.

O(n)	Jumlah operasi tumbuh sebanding dengan ukuran data.	Linear search.
O(n log n)	Umum pada algoritma pengurutan efisien.	Merge sort.
$O(n^2)$	Sering muncul pada dua perulangan bersarang.	Bubble sort pada kasus umum.

1.7.1 Mengabaikan Konstanta dan Suku Kecil

Dalam analisis asimtotik, perhatian utama diarahkan pada laju pertumbuhan saat n besar. Jika jumlah operasi dinyatakan sebagai $3n + 10$, maka pertumbuhan dominannya tetap linear sehingga ditulis $O(n)$. Jika jumlah operasi $n^2 + 5n + 20$, suku n^2 menjadi dominan sehingga ditulis $O(n^2)$. Penyederhanaan ini membantu membandingkan algoritma tanpa terikat pada detail perangkat.

1.7.2 Big-O Bukan Waktu Eksekusi dalam Detik

Big-O tidak mengatakan bahwa algoritma $O(n)$ selalu lebih cepat pada setiap input dibanding algoritma $O(n^2)$. Untuk input sangat kecil, konstanta, implementasi, dan perangkat dapat memengaruhi hasil. Namun, ketika ukuran input terus bertambah, pola pertumbuhan menjadi semakin penting. Pembahasan lebih lanjut tentang best case, average case, dan worst case akan diperluas pada pertemuan lanjutan.

1.8. Python sebagai Alat Implementasi

Python digunakan dalam mata kuliah ini karena sintaksnya relatif ringkas sehingga mahasiswa dapat lebih fokus pada konsep struktur data dan algoritma. Python bukan tujuan utama pembelajaran. Setiap implementasi akan selalu dikaitkan dengan representasi data, operasi yang dilakukan, dan kompleksitasnya.

1.8.1 Persiapan Lingkungan Implementasi Python

Praktik pada mata kuliah ini menggunakan *Google Colab* sebagai lingkungan utama. Pilihan ini memudahkan mahasiswa karena Python dapat dijalankan melalui *browser* tanpa instalasi awal. Notebook juga dapat disimpan secara daring sehingga contoh kode dari kelas dapat dibuka kembali pada pertemuan berikutnya.

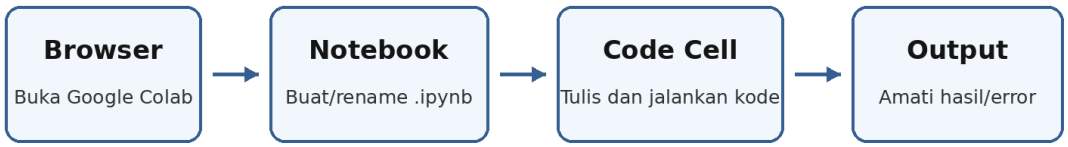
Komponen	Keterangan
Laptop/PC	Digunakan untuk menulis dan menjalankan kode.
Browser modern	Google Chrome, Microsoft Edge, Firefox, atau Safari.
Akun Google	Digunakan untuk membuka dan menyimpan *notebook* Google Colab.
Koneksi internet	Diperlukan ketika menggunakan Google Colab.

Notebook `.ipynb`	Berkas kerja yang berisi teks penjelasan, kode, dan hasil eksekusi.

Catatan: mahasiswa yang sudah terbiasa menggunakan Python lokal, VS Code, Jupyter Notebook, atau PyCharm tetap dapat menggunakannya. Namun, contoh praktikum di modul ini ditulis agar dapat langsung dijalankan di Google Colab.

1.8.2 Memulai Google Colab Langkah demi Langkah

1. Buka *browser* dan akses **https://colab.research.google.com**.
2. Masuk menggunakan akun Google.
3. Pilih **New notebook** untuk membuat notebook baru.
4. Ubah nama notebook, misalnya **SDA_Pertemuan_01_Nama_NIM.ipynb**.
5. Klik sebuah *code cell*, kemudian tuliskan kode Python.
6. Jalankan *cell* dengan tombol ▶ atau tekan **Shift + Enter**.
7. Perhatikan hasil yang muncul di bawah *cell*. Jika terjadi kesalahan, baca pesan *error*, perbaiki kode, lalu jalankan kembali.
8. Notebook akan tersimpan pada Google Drive. Pastikan perubahan terakhir sudah tersimpan sebelum menutup halaman.



Gambar 1.5 Alur dasar praktik Python menggunakan Google Colab
Sumber: Diolah oleh penulis.

1.8.3 Uji Lingkungan Pertama

Setelah notebook terbuka, jalankan kode berikut untuk memastikan lingkungan Python siap digunakan. Kode ini tidak berkaitan langsung dengan struktur data; tujuannya hanya memastikan bahwa mahasiswa memahami cara menulis kode, menjalankan *cell*, dan membaca keluaran.

Contoh 1.1 Uji lingkungan Python

```
print("Struktur Data dan Algoritma")
print("Python siap digunakan")
```

Keluaran yang diharapkan:

```
Struktur Data dan Algoritma
Python siap digunakan
```

Fungsi `print()` digunakan untuk menampilkan nilai atau teks ke layar. Teks diapit tanda kutip karena diperlakukan sebagai data bertipe string. Jika keluaran tersebut muncul tanpa pesan kesalahan, lingkungan implementasi sudah siap digunakan.

Untuk melihat versi Python yang sedang digunakan, jalankan kode berikut. Informasi versi tidak perlu dihafal, tetapi berguna ketika mahasiswa perlu memastikan lingkungan yang digunakan pada praktikum.

```
import sys
print(sys.version)
```

Perintah `import sys` memuat modul sistem Python. Baris kedua membaca informasi versi Python melalui `sys.version`. Pada pertemuan ini, mahasiswa cukup memahami bahwa `import` digunakan ketika program membutuhkan fungsi atau komponen dari modul lain.

1.8.4 Sintaks Minimum yang Dibutuhkan

Mahasiswa diasumsikan telah memperoleh dasar algoritma dan pemrograman. Pada pertemuan pertama, bagian ini hanya berfungsi sebagai penyegaran terhadap beberapa konstruksi Python yang akan sering digunakan.

Konstruksi	Peran	Contoh
Variabel	Menyimpan sebuah nilai.	`jumlah = 10`
list	Menyimpan sekumpulan nilai berurutan.	`data = [10, 20, 30]`
Indeks	Menunjuk posisi elemen pada *list*.	`data[0]`
`if`	Memilih aksi berdasarkan kondisi.	`if nilai > 70:`
`for`	Mengulangi proses pada sekumpulan data.	`for x in data:`
`def`	Mendefinisikan fungsi.	`def cari(...):`
`class`	Mendefinisikan tipe/objek yang memiliki data dan operasi.	`class Node:`

Contoh 1.2 Variabel, list, dan indeks

```
nilai = [80, 75, 90, 85]
print(nilai)
print(nilai[0])
print(nilai[-1])
print(len(nilai))
```

Bagian Kode	Penjelasan
`nilai = [80, 75, 90, 85]`	Membuat sebuah *list* berisi empat nilai.
`print(nilai)`	Menampilkan seluruh isi *list*.
`nilai[0]`	Mengambil elemen pertama. Indeks Python dimulai dari 0.
`nilai[-1]`	Mengambil elemen terakhir menggunakan indeks negatif.

`len(nilai)`	Menghitung jumlah elemen pada *list*.

Keluaran yang diharapkan:

```
[80, 75, 90, 85]
80
85
4
```

Contoh ini penting karena banyak struktur data pada pertemuan awal direpresentasikan sebagai sekumpulan elemen. Mahasiswa perlu terbiasa membaca posisi elemen, jumlah elemen, dan perubahan isi data sebelum mempelajari operasi yang lebih kompleks.

Contoh 1.3 Perulangan pada sekumpulan data

```
data = [10, 20, 30, 40]

for nilai in data:
    print(nilai)
```

Bagian Kode	Penjelasan
`data = [10, 20, 30, 40]`	Membuat *list* yang berisi empat angka.
`for nilai in data:`	Mengambil elemen satu per satu dari awal hingga akhir. Variabel `nilai` berisi elemen yang sedang diproses.
`print(nilai)`	Menampilkan elemen yang sedang diproses. Baris ini menjorok ke dalam karena menjadi bagian dari *loop*.

Keluaran yang diharapkan:

```
10
20
30
40
```

Kode tersebut melakukan *traversal* sederhana: setiap elemen pada *list* dikunjungi tepat satu kali. Jika terdapat n elemen, jumlah iterasi bertambah sebanding dengan n, sehingga pola kerjanya bersifat linear atau O(n). Perhatikan juga **indentasi** pada `print(nilai)`. Dalam Python, indentasi menentukan bahwa perintah tersebut berada di dalam blok `for`.

1.8.5 Fungsi, Parameter, dan Nilai Kembali

Fungsi digunakan untuk mengelompokkan langkah algoritma agar dapat dipanggil kembali. Pada contoh pencarian linear, fungsi menerima dua parameter: `data` sebagai kumpulan elemen dan `target` sebagai nilai yang ingin dicari. Fungsi kemudian mengembalikan posisi target jika ditemukan.

Contoh 1.4 Fungsi pencarian linear sederhana

```
def cari(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i
    return -1
```

Bagian Kode	Penjelasan
`def cari(data, target):`	Mendefinisikan fungsi bernama `cari` dengan dua parameter.
`for i in range(len(data)):`	Membentuk indeks 0 sampai `len(data)-1` dan memeriksa elemen satu per satu.
`if data[i] == target:`	Membandingkan elemen pada posisi `i` dengan target.
`return i`	Menghentikan fungsi dan mengembalikan indeks ketika target ditemukan.
`return -1`	Dijalankan jika seluruh elemen sudah diperiksa tetapi target tidak ditemukan.

Coba panggil fungsi tersebut menggunakan kode berikut:

```
data = [12, 20, 7, 30]
print(cari(data, 7))
print(cari(data, 99))
```

Keluaran yang diharapkan:

```
2
-1
```

Target `7` ditemukan pada indeks 2, sedangkan `99` tidak ditemukan sehingga fungsi mengembalikan -1. Pada kondisi terburuk, fungsi harus memeriksa seluruh n elemen. Karena jumlah pemeriksaan bertambah sebanding dengan ukuran data, pencarian linear memiliki kompleksitas waktu O(n). Pada Pertemuan 2, pola ini akan dibandingkan dengan *binary search*.

1.8.6 Membaca Pesan Kesalahan saat Praktik

Kesalahan saat menulis kode merupakan bagian normal dari praktik pemrograman. Tujuan mahasiswa bukan menghindari seluruh *error*, tetapi mampu membaca pesan yang diberikan Python dan menemukan bagian kode yang perlu diperbaiki. Beberapa kesalahan yang sering muncul pada pertemuan awal ditunjukkan pada tabel berikut.

Pesan Error	Makna Umum	Langkah Awal
`SyntaxError`	Struktur penulisan kode tidak sesuai aturan Python.	Periksa tanda titik dua, kurung, atau penulisan perintah.
`IndentationError`	Indentasi blok kode tidak konsisten.	Rapikan spasi pada kode di bawah `for`, `if`, atau `def`.
`NameError`	Nama variabel/fungsi belum pernah dibuat atau salah ketik.	Periksa ejaan dan urutan eksekusi *cell*.

`TypeError`	Operasi diterapkan pada tipe data yang tidak sesuai.	Periksa tipe nilai yang digunakan dalam operasi.
`IndexError`	Program mencoba mengakses indeks yang tidak tersedia.	Periksa panjang *list* dan nilai indeks.

Pada Google Colab, baris terakhir pesan kesalahan biasanya menyebut jenis *error*. Baca jenis tersebut terlebih dahulu, kemudian lihat nomor baris dan potongan kode yang ditandai. Setelah memperbaiki kode, jalankan kembali *cell*.

1.8.7 Class sebagai Sarana Membangun Struktur Data

Beberapa struktur data akan diimplementasikan menggunakan `class` agar data dan operasi dapat dikemas dalam satu objek. Mahasiswa tidak dituntut mempelajari seluruh konsep pemrograman berorientasi objek pada mata kuliah ini; penggunaan class dibatasi pada kebutuhan implementasi struktur data.

Contoh 1.5 Bentuk awal node yang akan digunakan pada linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Bagian Kode	Penjelasan
`class Node:`	Mendefinisikan tipe objek baru bernama `Node`.
`def __init__(self, data):`	Konstruktor yang dijalankan ketika objek `Node` dibuat. Parameter `data` menerima nilai yang akan disimpan.
`self.data = data`	Menyimpan nilai ke atribut `data` milik objek.
`self.next = None`	Menyiapkan atribut `next`. Nilai `None` berarti node belum menunjuk ke node lain.

Untuk mencoba kelas tersebut, jalankan kode berikut pada *cell* baru:

```
node1 = Node(10)
print(node1.data)
print(node1.next)
```

Keluaran yang diharapkan:

```
10
None
```

Mahasiswa belum perlu menguasai seluruh konsep pemrograman berorientasi objek. Pada mata kuliah ini, `class` digunakan terutama untuk membentuk struktur seperti *node*, *linked list*, *tree*, dan struktur lain yang memerlukan data beserta hubungan antarelemen.

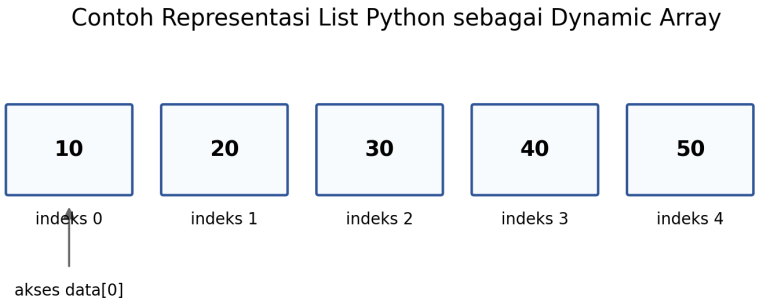
Praktik Mandiri 1.1

Lakukan latihan berikut di Google Colab untuk memastikan persiapan praktikum sudah selesai:

1. Buat notebook bernama **SDA_Pertemuan_01_Nama_NIM.ipynb**.
2. Jalankan Contoh 1.1 hingga Contoh 1.5 secara berurutan.
3. Pada Contoh 1.2, tambahkan satu nilai baru ke dalam `nilai`, kemudian tampilkan kembali panjang *list*.
4. Pada Contoh 1.4, ganti isi `data` dan `target`, kemudian amati indeks yang dikembalikan.
5. Pada Contoh 1.5, buat dua objek `Node` dengan data berbeda dan tampilkan nilai masing-masing objek.
6. Simpan notebook. Notebook ini akan menjadi titik awal praktik pada pertemuan berikutnya.

1.9. Struktur Data Bawaan Python: Gunakan dengan Memahami Konsep

Python menyediakan beberapa tipe data bawaan yang sangat berguna. Namun, penggunaan tipe bawaan tidak boleh menggantikan pemahaman terhadap struktur data yang mendasarinya. Dalam praktik, kita dapat menggunakan fitur bawaan setelah memahami karakteristik operasinya.



Gambar 1.6 Contoh list Python sebagai dynamic array

Sumber: Diolah oleh penulis.

Python	Karakteristik Konseptual	Contoh Penggunaan
list	Dynamic array; memiliki indeks dan ukuran dapat berubah.	Daftar nilai, implementasi stack sederhana.
tuple	Urutan elemen yang tidak dapat diubah setelah dibuat.	Data yang bersifat tetap.
dict	Pemetaan key-value berbasis konsep hash table.	Pencarian data berdasarkan kunci.
set	Kumpulan elemen unik berbasis hashing.	Pemeriksaan keanggotaan dan eliminasi duplikasi.
deque	Double-ended queue dari modul collections.	Queue dengan operasi dari kedua ujung.

1.9.1 Catatan Penting tentang Python list

Pada Python, `list` bukan array statis seperti pada C. Implementasinya lebih dekat dengan *dynamic array*. Karena itu, ukuran list dapat bertambah dan berkurang. Walaupun demikian, list tetap sangat berguna untuk menjelaskan akses berdasarkan indeks dan berbagai operasi dasar array.

Contoh 1.6 Beberapa operasi dasar pada list

```
data = [10, 20, 30]
print(data[1])      # akses indeks
data.append(40)    # menambah di akhir
data.insert(0, 5)  # menyisipkan di awal
```

Operasi	Makna
`data[1]`	Mengakses elemen indeks 1, yaitu nilai 20.
`append(40)`	Menambahkan 40 pada bagian akhir *list*.
`insert(0, 5)`	Menyisipkan 5 pada indeks 0. Elemen lain bergeser ke kanan.
`pop()`	Menghapus dan mengembalikan elemen terakhir.

Mahasiswa perlu menguji setiap baris satu per satu dan mencetak isi `data` setelah operasi dilakukan. Cara ini membantu melihat bahwa perubahan satu baris kode dapat mengubah posisi dan jumlah elemen pada struktur data.

Pada pertemuan berikutnya, setiap operasi akan dianalisis dari sisi perilaku dan kompleksitasnya. Mahasiswa perlu membedakan antara “mudah ditulis” dan “efisien dilakukan”. Satu baris kode Python dapat menyembunyikan proses internal yang cukup kompleks.

1.10. Studi Kasus Awal: Mengelola Data Mahasiswa

Untuk melihat bagaimana pemilihan struktur data muncul dari kebutuhan, gunakan contoh sederhana sistem yang menyimpan NIM dan nama mahasiswa. Kita mulai dari solusi yang mudah dipahami, lalu melihat keterbatasannya.

1.10.1 Solusi dengan List

Contoh 1.7 Pencarian NIM menggunakan list

```
mahasiswa = [
    ('22001', 'Alya'),
    ('22002', 'Bima'),
    ('22003', 'Citra')
]

def cari_nim(data, nim):
    for item in data:
        if item[0] == nim:
            return item
    return None

print(cari_nim(mahasiswa, '22003'))
```

Pendekatan ini mudah dipahami. Namun, untuk mencari satu NIM, program mungkin harus memeriksa banyak elemen. Jika terdapat n mahasiswa, pencarian linear memiliki kompleksitas O(n) pada kasus terburuk.

1.10.2 Solusi dengan Dictionary

Contoh 1.8 Pencarian NIM menggunakan dictionary

```
mahasiswa = {
    '22001': 'Alya',
    '22002': 'Bima',
    '22003': 'Citra'
}

print(mahasiswa.get('22003'))
```

Dictionary memungkinkan data diakses menggunakan kunci. Secara konseptual, struktur ini terkait dengan hash table. Rata-rata, pencarian berdasarkan kunci dapat dilakukan sangat cepat. Akan tetapi, struktur ini juga mempunyai biaya memori dan karakteristik internal yang berbeda dari list. Hash table akan dibahas secara khusus pada pertemuan lanjutan dalam semester ini.

1.10.3 Pelajaran dari Studi Kasus

Tidak ada struktur data yang selalu terbaik untuk semua masalah. List cocok ketika urutan penting dan data perlu ditelusuri berurutan. Dictionary cocok ketika pencarian berdasarkan kunci menjadi kebutuhan utama. Pertanyaan penting bukan “struktur data apa yang paling bagus?”, tetapi “struktur data apa yang paling sesuai dengan pola operasi pada masalah ini?”.

1.11. Cara Memilih Struktur Data yang Sesuai

Pemilihan struktur data sebaiknya dilakukan secara sistematis. Mahasiswa dapat menggunakan beberapa pertanyaan berikut sebelum mulai menulis kode.

1. Apa bentuk data yang akan disimpan? Apakah berupa urutan, pasangan key-value, hierarki, atau jaringan hubungan?
2. Operasi apa yang paling sering dilakukan? Apakah akses indeks, pencarian, penambahan, penghapusan, atau traversal?
3. Apakah urutan elemen harus dipertahankan?
4. Apakah data sering berubah ukuran?
5. Apakah waktu eksekusi atau penggunaan memori menjadi batas penting?
6. Apakah terdapat aturan khusus, misalnya LIFO atau FIFO?

Kebutuhan Dominan	Kandidat Struktur Data	Alasan Awal
Akses cepat berdasarkan posisi	Array / list	Indeks memberi akses langsung ke posisi tertentu.
Sering menambah/menghapus node melalui relasi	Linked list	Hubungan node dapat diubah tanpa menggeser seluruh elemen.
Aksi terakhir diproses lebih dahulu	Stack	Mengikuti prinsip LIFO.
Pelayanan sesuai urutan kedatangan	Queue	Mengikuti prinsip FIFO.
Pencarian berdasarkan key	Hash table / dict	Akses berbasis kunci sangat efisien pada rata-rata kasus.
Hubungan hierarkis	Tree	Merepresentasikan parent-child secara natural.
Jaringan hubungan umum	Graph	Merepresentasikan vertex dan edge.

1.11.1 Trade-off

Trade-off berarti keuntungan pada satu aspek dapat diikuti biaya pada aspek lain. Struktur data yang memberikan pencarian cepat mungkin membutuhkan memori lebih besar. Struktur yang mudah untuk penyisipan mungkin tidak memberikan akses indeks secara langsung. Karena itu, pemilihan struktur data selalu terkait kebutuhan aplikasi dan pola penggunaan.

1.11.2 Hubungan dengan Pertemuan Selanjutnya

Pada Pertemuan 2 sampai 13, setiap struktur data dan algoritma akan dibahas dengan pola yang sama: konsep, representasi, operasi, implementasi Python, contoh kasus, dan analisis kompleksitas. Dengan pola tersebut, mahasiswa diharapkan tidak hanya mampu menulis kode, tetapi juga mampu menjelaskan alasan pemilihan suatu solusi.

1.12. Kesalahan Umum pada Tahap Awal

Beberapa kesalahan sering muncul ketika mahasiswa baru mempelajari struktur data dan algoritma. Mengenali kesalahan tersebut sejak awal akan membantu proses belajar pada pertemuan berikutnya.

- Menganggap struktur data hanya sebagai sintaks bahasa pemrograman. Struktur data adalah konsep; implementasinya dapat berbeda pada Python, Java, C++, atau bahasa lain.
- Memilih struktur data karena paling mudah ditulis, tanpa mempertimbangkan operasi utama yang dibutuhkan.
- Menganggap fungsi bawaan selalu O(1). Satu baris kode dapat menjalankan banyak operasi di balik layar.
- Menghafal Big-O tanpa memahami pola pertumbuhan dan kaitannya dengan ukuran input.
- Menilai efisiensi hanya dari satu kali pengukuran waktu program.
- Berfokus pada kode sebelum memahami masalah dan representasi data.

1.13. Rangkuman

Struktur data mengatur bagaimana data disimpan dan dihubungkan, sedangkan algoritma menentukan langkah untuk memproses data. Pemilihan struktur data harus mempertimbangkan operasi utama dan efisiensi, bukan hanya kemudahan menulis kode. Pada pertemuan ini mahasiswa juga menyiapkan Google Colab sebagai lingkungan praktik, mengenali *list*, indeks, *loop*, fungsi, `class`, serta cara membaca pesan *error*. Python digunakan sebagai alat untuk mengamati perilaku struktur data dan algoritma secara langsung.

1.14. Latihan dan Evaluasi

1. Jelaskan dengan kalimat sendiri perbedaan antara data, struktur data, algoritma, dan program.
2. Sebutkan masing-masing dua contoh struktur data linear dan non-linear beserta satu contoh penggunaan.
3. Mengapa konsep ADT berguna ketika sebuah struktur data dapat memiliki lebih dari satu implementasi?
4. Untuk sistem antrean layanan, mengapa queue lebih sesuai daripada stack? Jelaskan prinsip operasinya.
5. Jelaskan secara intuitif perbedaan pertumbuhan O(1), O(log n), O(n), dan $O(n^2)$.
6. Buat program Python sederhana yang menyimpan lima angka ke dalam list, menampilkan seluruh elemen, lalu mencari satu nilai menggunakan pencarian linear.
7. Berikan satu contoh situasi ketika dictionary lebih sesuai daripada list dan jelaskan alasannya.
8. Jelaskan langkah yang Anda lakukan untuk menyiapkan Google Colab hingga berhasil menjalankan kode Python pertama.

9. Jalankan fungsi `cari()` dengan data minimal 10 elemen. Catat target yang dicari, indeks hasil, dan jelaskan mengapa pada kasus tertentu fungsi harus memeriksa hampir seluruh elemen.

Latihan Diskusi

Sebuah aplikasi kampus harus menyimpan 50.000 data mahasiswa. Sistem sering melakukan pencarian berdasarkan NIM, tetapi hanya sesekali menampilkan seluruh mahasiswa secara berurutan. Diskusikan struktur data awal yang paling masuk akal dan jelaskan operasi apa yang menjadi pertimbangan utama. Pada tahap ini tidak diperlukan implementasi penuh.

DAFTAR PUSTAKA

[1]. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2]. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. Data Structures and Algorithms in Python. John Wiley & Sons, 2013.

[3]. Miller, B. N., & Ranum, D. L. Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates, 2011.

[4]. Bhargava, A. Y. Grokking Algorithms, Second Edition: An Illustrated Guide for Programmers and Other Curious People. Manning, 2024.

[5]. Python Software Foundation. The Python Tutorial: Data Structures. Dokumentasi resmi Python, https://docs.python.org/3/tutorial/datastructures.html.

[6]. Google. Google Colaboratory. https://colab.research.google.com/.